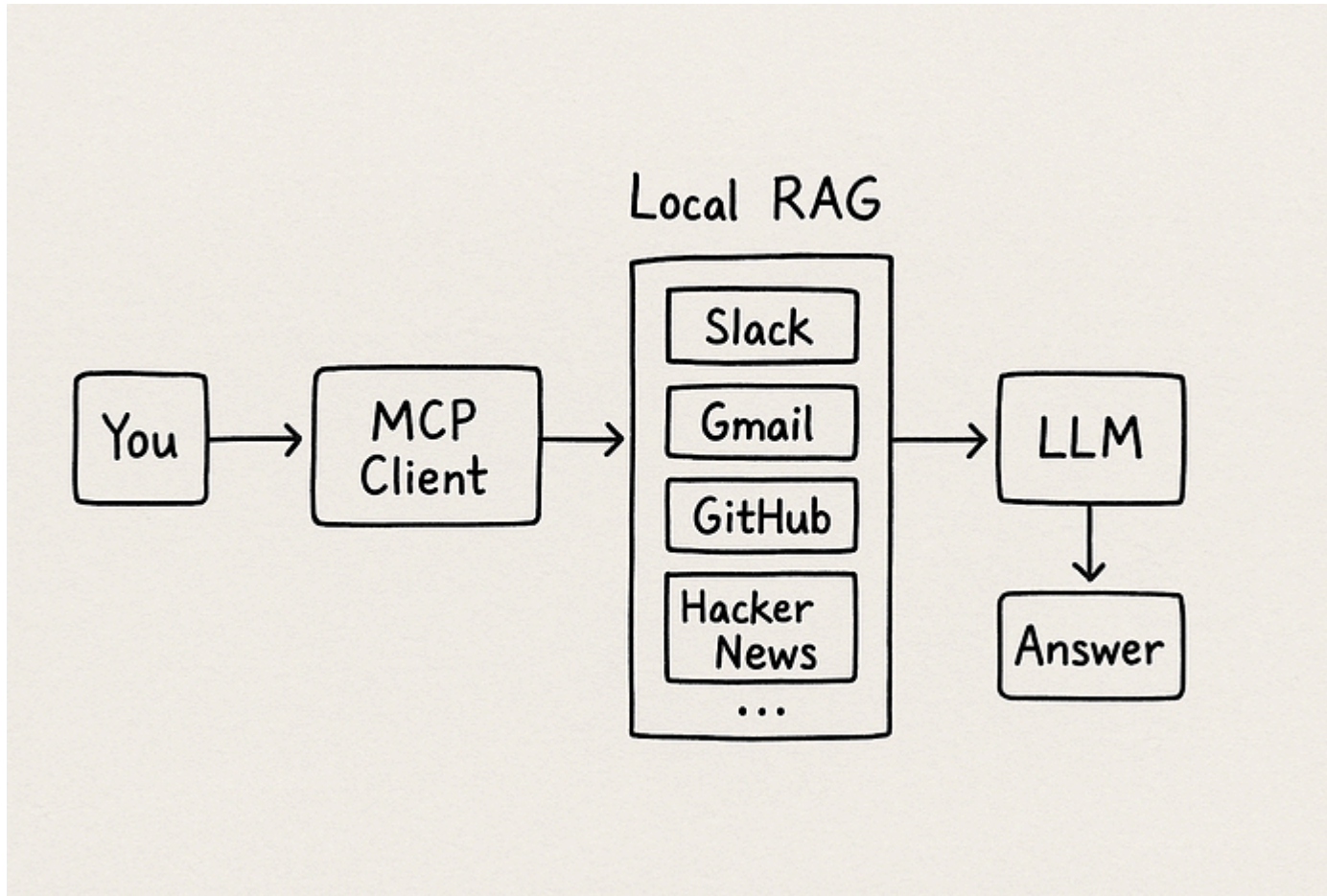


[< Go to the original](#)

Building a Local MCP-Powered RAG That Can Talk to 200+ Data Sources

**Civil Learning**

Follow

Coding Nexus a11y-light ~9 min read · August 9, 2025 (Updated: August 12, 2025) · Free: No

You've got emails in Gmail.

Conversations in Slack.

Code in GitHub.

Reports in Google Drive.

Maybe even some random analytics dashboard that nobody remembers the password for.

If you're trying to build a **Retrieval-Augmented Generation** (RAG) setup, that's a nightmare. You can't just connect to *one* data source and call it a day, unless you're happy with half-baked answers.

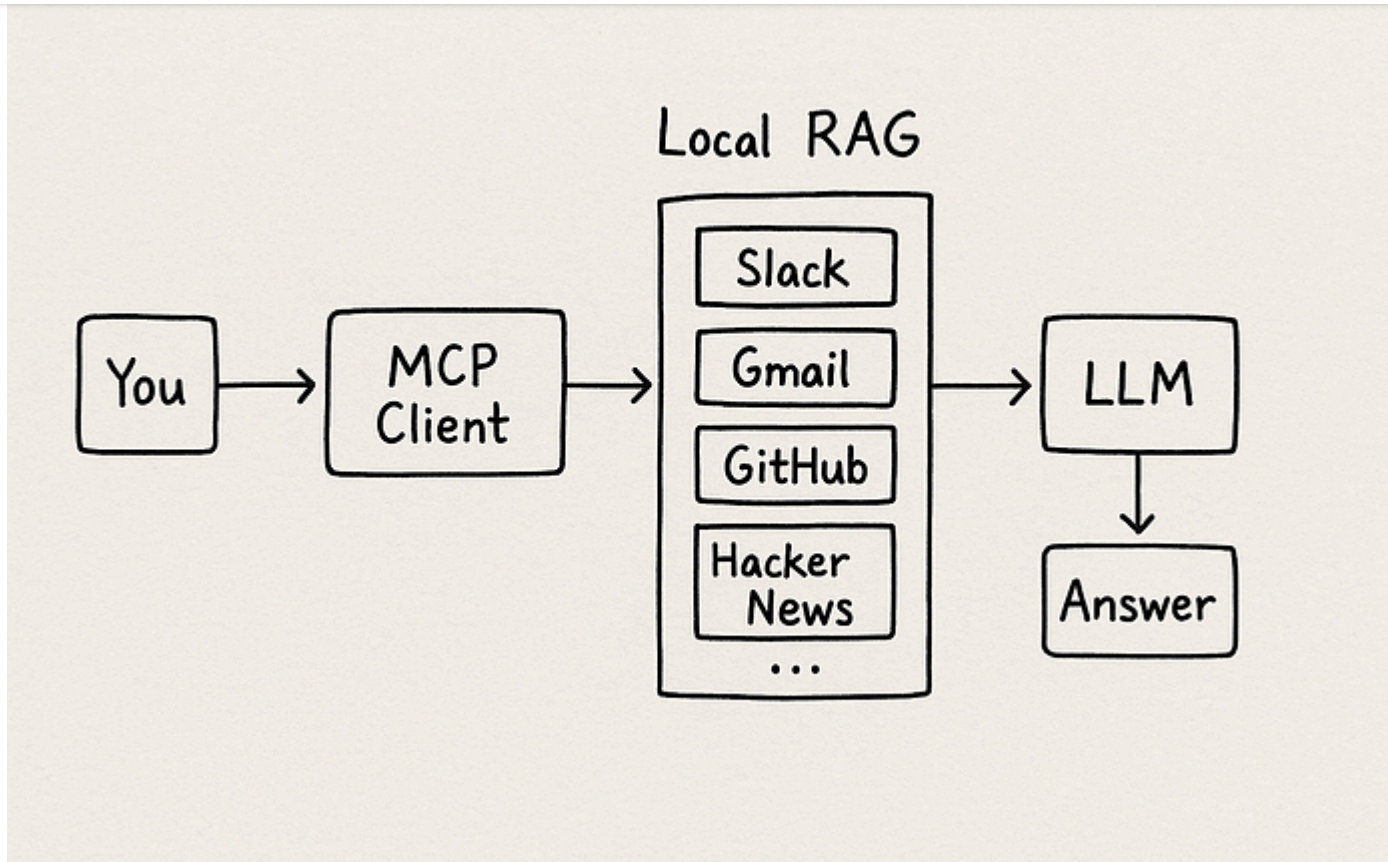
The big players already know this:

- ~~Google Bakes it into Vertex AI Search.~~

- AWS has their own in Amazon Q Business.

Nice if you're all-in on their ecosystem. But what if you want to do it **your way**, locally, with zero cloud dependency?

Well... let's build one.



Rough hand-drawn-style workflow diagram generated through AI

What We're Making

We're going to make a local AI setup that can query over 200+ different sources — all from a single chat interface — using:

- [mcp-use](#) (a handy tool from YC S25) for the local MCP client.

• Channel to run open-source LLMs locally.

The flow is pretty straightforward once you see it:

1. You type a question into your chat box.
2. The MCP client passes it to a MindsDB MCP server.
3. MindsDB figures out which data source(s) you're actually asking about.
4. It queries them, sends the results to your LLM, and — voilà — you get an answer with actual context.

If you like pictures more than text, it's basically:

Copy

You → MCP Client → MindsDB MCP Server → Your Data Sources → LLM → Answer

Step 1 — Run MindsDB Locally

We'll run MindsDB in Docker. Open your terminal and run:

Copy

That's it. No signup. No cloud. You're now running MindsDB on your machine.

Step 2 — Get the MindsDB GUI

Pop open your browser and head to:

Copy

```
http://localhost:47334
```

You should see the MindsDB editor — kind of like a SQL workbench, but with magic built in. From here, you can connect to hundreds of different sources: Slack, Gmail, Salesforce, MySQL, you name it.

Step 3 — Connect Some Data

Here's me connecting a few just to show how dead simple it is:

Copy

```
-- Slack  
CREATE DATABASE slack_db
```

```
token = your-stack-api-token  
};  
  
-- Gmail  
CREATE DATABASE gmail_db  
WITH ENGINE = "gmail",  
PARAMETERS = {  
  "credentials": "gmail_credentials.json"  
};  
  
-- GitHub  
CREATE DATABASE github_db  
WITH ENGINE = "github",  
PARAMETERS = {  
  "token": "your-github-token"  
};  
  
-- Hacker News  
CREATE DATABASE hn_db  
WITH ENGINE = "hacker_news";
```

After this, you can run regular SQL queries like:

Copy

```
SELECT * FROM gmail_db.inbox WHERE is_unread = true;
```

Pretty wild.

Step 4 — MCP Server Config

Alright, MindsDB can talk to our data, but now we want it to speak MCP so our AI agent can use it. We make a little JSON file for that:

Copy

```
{
  "servers": {
    "mindsdb": {
      "url": "http://localhost:47334",
      "tools": ["list_databases", "query"]
    }
  }
}
```

Now our MCP client knows where the MindsDB server lives and what tools it has.

Step 5 — Local MCP Client + Ollama

This part is actually fun — it's only a few lines in Python.

Copy


```
# 1. Load MCP client config
client = MCPClient.from_config("mcp_config.json")
# 2. Connect local LLM
llm = ChatModel("llama3")
# 3. Create agent
agent = llm.bind(client)
# 4. Run a query
response = agent.ask("Show me all unread Slack messages from this week.")
print(response)
```

It's almost too easy. You can literally swap the LLM by changing "llama3" to "mistral" or "gemma2" .

Step 6 — A Quick UI in Streamlit

If you want a nice browser-based chat box (and who doesn't):

Copy

```
import streamlit as st

st.title("MCP-Powered RAG")
query = st.text_input("Ask me anything:")
if st.button("Run"):
```

Run it:

Copy

```
streamlit run app.py
```

Type in "List all GitHub issues assigned to me" and watch the magic happen.

Here's a single Python file (`app.py`) that brings the whole Local-RAG workflow together in one place — from the MCP client, through MindsDB tools, to a local LLM (Ollama), all wrapped in a Streamlit UI.

Install (one-time)

Copy

```
# create a venv if you like, then:
pip install streamlit requests
# The following two are provided as-is; install if you have them
pip install mcp_use ollama
# if these packages are not available on pip in your environment,
# follow the mcp_use / ollama install docs and adapt the imports below.
```

Copy

```
"""
```

```
app.py
```

```
Single-file Streamlit app that demonstrates a Local RAG flow:
```

```
You (UI) -> MCP client -> MindsDB (list_databases / query) -> Ollama LLM -> Ans
```

```
Notes:
```

- Requires MindsDB running on `http://localhost:47334` (MindsDB Docker).
- Requires local Ollama / LLM accessible via ollama Python bindings.
- The `mcp_use` Python client is assumed; if your environment differs, adapt the small wrapper functions below to call the correct methods.

```
Run:
```

```
streamlit run app.py
```

```
"""
```

```
import json
```

```
import time
```

```
import traceback
```

```
from typing import Any, Dict, List, Optional
```

```
import streamlit as st
```

```
import requests
```

```
# Attempt imports for mcp_use and ollama. If missing, the app will still start
```

```
try:
```

```
    from mcp_use import MCPClient # expected in your environment per the earl
```

```
    _HAS_MCP_USE = True
```

```

_HAS_MCP_USE = False

try:
    # Ollama python client shape differs by versions; this mirrors earlier usage
    from ollama import ChatModel
    _HAS_OLLAMA = True
except Exception:
    ChatModel = None
    _HAS_OLLAMA = False

# -----
# Utility / Wrappers
# -----

DEFAULT_MCP_CONFIG = {
    "servers": {
        "mindsdb": {
            "url": "http://localhost:47334",
            "tools": ["list_databases", "query"]
        }
    }
}

def ensure_mcp_config_file(path: str = "mcp_config.json"):
    """Write a default MCP config if file not present."""
    try:
        with open(path, "x") as f:
            json.dump(DEFAULT_MCP_CONFIG, f, indent=2)

```

```
pass

class SimpleMCPWrapper:
    """
    Small compatibility wrapper around mcp_use.MCPClient to provide
    list_databases() and query() calls in a forgiving way.
    """

    def __init__(self, config_path: str = "mcp_config.json"):
        self.config_path = config_path
        self.client = None
        if _HAS_MCP_USE and MCPClient is not None:
            try:
                self.client = MCPClient.from_config(config_path)
            except Exception as e:
                # Some versions might have a different constructor – attempt a
                try:
                    self.client = MCPClient(config_path)
                except Exception as e2:
                    print("Failed to init MCPClient from library. Will fallback")
                    print("mcp_use error:", e, e2)

        # Also store MindsDB base URL for HTTP fallback
        with open(config_path, "r") as f:
            cfg = json.load(f)
        try:
            self.mindsdb_url = cfg["servers"]["mindsdb"]["url"].rstrip("/")
        except Exception:
```

```

def list_databases(self) -> List[str]:
    """Return list of databases / sources. Try client, then HTTP fallback."""
    # Try MCP client call
    if self.client:
        # try common method names
        for name in ("list_databases", "list_databases_tool", "list_tools",
                    "list_data_sources", "list_data_sources_tool"):
            fn = getattr(self.client, name, None)
            if callable(fn):
                try:
                    out = fn()
                    # if returned a dict with 'databases' or similar, normalize
                    if isinstance(out, dict):
                        if "databases" in out:
                            return out["databases"]
                        # try to parse other shapes
                        return list(out.keys())
                    if isinstance(out, (list, tuple)):
                        return list(out)
                except Exception:
                    pass
        # try a generic 'call' or 'invoke' interface if present
        for name in ("call_tool", "invoke_tool", "call"):
            fn = getattr(self.client, name, None)
            if callable(fn):
                try:
                    # some mcp clients expect server/tool names
                    out = fn("mindsdb", "list_databases", {})
                    return out
                except TypeError:

```

```

        return list(out)

        if isinstance(out, dict) and "databases" in out:
            return out["databases"]
    except Exception:
        pass

# HTTP fallback to MindsDB REST (best-effort)
# MindsDB doesn't have a single standardized public "list databases" endpoint
# but we can attempt to call the schema or connectors endpoints.
try:
    # attempt common API endpoints for MindsDB
    resp = requests.get(f"{self.mindsdb_url}/api/databases", timeout=60)
    if resp.ok:
        payload = resp.json()
        # try common shapes
        if isinstance(payload, dict):
            if "data" in payload and isinstance(payload["data"], list):
                return [d.get("name") or d.get("database") or str(d) for d in payload["data"]]
            return list(payload.keys())
        if isinstance(payload, list):
            return [p.get("name") if isinstance(p, dict) else str(p) for p in payload]
except Exception:
    pass

# If we reach here, return a helpful message
return ["(unable to enumerate – check MCP client or MindsDB)"]

def query(self, query_text: str, max_rows: int = 50) -> Dict[str, Any]:
    """

```

```
# Try MCP client
if self.client:
    for name in ("query", "run_query", "invoke_query", "call_tool"):
        fn = getattr(self.client, name, None)
        if callable(fn):
            try:
                # try several argument shapes
                try:
                    out = fn("mindsdb", "query", {"query": query_text,
except TypeError:
                    out = fn(query_text)
                return {"source": "mcp_client", "result": out}
            except Exception:
                # continue trying other method names
                pass

# Fallback: try MindsDB query HTTP endpoint (best-effort use of /api/sc
try:
    # Some MindsDB versions expose /api/sql or /api/query; try both
    for endpoint in ("/api/sql", "/api/query", "/api/query/execute", "/
        try:
            resp = requests.post(self.mindsdb_url + endpoint, json={"q
        except Exception:
            continue
        if not resp.ok:
            continue
    try:
        data = resp.json()
```



```

        return { "source": "mindsdb_http", "result": data }

    except Exception:
        pass

    # If all fail:
    return {"source": "error", "error": "Unable to run query. Check MCP cl

class OllamaWrapper:
    """Minimal wrapper to send a prompt to Ollama (local LLM)."""

    def __init__(self, model_name: str = "llama3"):
        self.model_name = model_name
        self.model = None
        if _HAS_OLLAMA and ChatModel is not None:
            try:
                # API varies – some libs are ChatModel(name=...), other use cl
                try:
                    self.model = ChatModel(model_name)
                except Exception:
                    # try constructing differently
                    self.model = ChatModel(model=model_name)
            except Exception as e:
                print("Failed to init ollama.ChatModel:", e)
                self.model = None

    def chat(self, prompt: str, max_tokens: int = 512) -> str:
        # Try python binding
        if self.model:

```

```
    if isinstance(out, str):
        try:
            out = fn(prompt)
            # many wrappers return complex object; normalize to str
            if isinstance(out, (str,)):
                return out
            if isinstance(out, dict) and "text" in out:
                return out["text"]
            # try to stringify
            return str(out)
        except Exception:
            continue

# Fallback: attempt to use local Ollama HTTP (if Ollama daemon listening)
try:
    ollama_url = "http://localhost:11434/api/generate"
    payload = {"model": self.model_name, "prompt": prompt}
    resp = requests.post(ollama_url, json=payload, timeout=20)
    if resp.ok:
        data = resp.json()
        # shape will vary; try to extract common fields
        if isinstance(data, dict):
            if "text" in data:
                return data["text"]
            if "response" in data:
                return str(data["response"])
        return str(data)
except Exception:
    pass
```

```
# -----  
# Build Streamlit UI  
# -----  
  
st.set_page_config(page_title="Local RAG (MCP + MindsDB + Ollama)", layout="wide")  
st.title("Local RAG – one-frame demo (MCP → MindsDB → Ollama)")  
  
st.markdown(  
    """  
    This demo expects:  
    – MindsDB running locally (Docker) at `http://localhost:47334`  
    – A local Ollama LLM (or Ollama HTTP endpoint at http://localhost:11434)  
    – The `mcp_use` and `ollama` python libs if you want direct bindings.  
  
    If any piece is missing, the UI will still let you run queries via HTTP fallback.  
    """  
)  
  
# ensure default config exists  
ensure_mcp_config_file("mcp_config.json")  
  
col1, col2 = st.columns([1, 2])  
  
with col1:  
    st.subheader("MCP / MindsDB")  
    st.write("Edit the MCP config (JSON):")  
    mcp_config_text = st.text_area("mcp_config.json", value=json.dumps(DEFAULT_CONFIG))
```

```
        f.write(mcp_config_text)
    st.success("Saved mcp_config.json")
    time.sleep(0.2)

if st.button("List data sources"):
    wrapper = SimpleMCPWrapper("mcp_config.json")
    try:
        dbs = wrapper.list_databases()
        st.write("Data sources / databases:")
        st.json(dbs)
    except Exception as e:
        st.error("Failed to list databases: " + str(e))
        st.text(traceback.format_exc())

st.markdown("----")
st.write("Run a raw federated SQL query against MindsDB (best-effort):")
raw_query = st.text_area("Raw SQL (example):", value="SELECT * FROM gmail_c")

if st.button("Run raw query"):
    wrapper = SimpleMCPWrapper("mcp_config.json")
    res = wrapper.query(raw_query)
    if res.get("source") == "error":
        st.error(res.get("error"))
    else:
        st.write("Result (source = %s)" % res.get("source"))
        st.json(res.get("result"))

with col2:
    st.subheader("Chat + RAG (Query → MindsDB tool → Ollama)")
```

```
model_name = st.text_input("Ollama model name:", value="llama3")
max_rows = st.slider("Max rows to fetch (federated query)", 1, 500, 50)

if st.button("Run RAG query") and user_q.strip():
    st.info("Running RAG flow...")

    # 1) Call MCP wrapper to run a mindsdb 'query' tool (best-effort)
    wrapper = SimpleMCPWrapper("mcp_config.json")
    # Very simple heuristics – in practice you'd call the MCP "query" tool
    # user text to SQL or asks MindsDB's SQL transformer. Here we just forward
    try:
        mindsdb_resp = wrapper.query(user_q, max_rows=max_rows)
    except Exception as e:
        st.error("MindsDB query step failed: " + str(e))
        mindsdb_resp = {"source": "error", "error": str(e)}

    if mindsdb_resp.get("source") == "error":
        st.error("MindsDB step couldn't run. See details below.")
        st.json(mindsdb_resp)
    else:
        st.success("MindsDB returned results (best-effort).")
        st.subheader("Raw data from MindsDB")
        st.json(mindsdb_resp.get("result"))

    # 2) Summarize / generate answer with local LLM
    st.subheader("LLM summary / answer")
    ollama = OllamaWrapper(model_name=model_name)
```

```

raw_text = mcp_sub_resp.get('result')
# convert complex payload into a compact string
try:
    raw_preview = json.dumps(raw_text, indent=2)[:4000]
except Exception:
    raw_preview = str(raw_text)[:4000]

prompt = (
    "You are a helpful assistant. The user asked:\n\n"
    f"\n\"{user_q}\"\\n\\n"
    "Here are results retrieved from federated data sources (truncated):\n\n"
    f"{raw_preview}\\n\\n"
    "Please provide a concise, clear and actionable answer that reflects the results."
)

# call chat model
llm_response = ollama.chat(prompt)
st.write(llm_response)

st.markdown("----")
st.caption("This example is intentionally defensive: it tries Python bindings to avoid
          \"Adapt the small wrappers to match the exact mcp_use / ollama APIs y")

```

Where This Leaves Us

- 200+ sources. One interface.

Freedium

ko-fi librepay patreon

-
- Easy to swap components — you can change your LLM, add/remove data sources, or even change the MCP client without breaking everything.

#rags #mcp-server #data #ai #model-context-protocol